Lab 1

Instructions for Lab Submission:

1. File Naming and Submission:
    a. Submit your solution as a zip file named **lab1_<ERP>.zip**, where <*ERP*> is your ERP number. For example, if your ERP number is *12345*, the file should be named *lab1_12345.zip*.
    b. Ensure that the zip file contains:
        i. C++ source files named as **task<number>.cpp** (e.g., **task1.cpp**).
        ii. No unnecessary files (such as executables, temporary files, or editor configs).
    c. Do not submit ***TEXT(.txt)*** files.
2. Code Organization:
    a. Write each task in its own **.cpp** file (as specified above).
    b. Ensure that your code is modular and logically separated into functions when specified.
3. Code Neatness:
    a. Ensure your code is well-formatted and easy to read. Use proper indentation and meaningful variable names.
    b. Include appropriate comments to explain the logic where necessary (especially for functions and complex sections of the code).
    c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
4. Code Compilation and Functionality:
    a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
    b. Double-check that the functionality of the program meets the requirements outlined in the task.
5. Additional Instructions:
    a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
    b. Make sure your code adheres to the principles of good software development (e.g., modular functions, minimal repetition, and no hardcoding).
    c. If you encounter any issues or require clarifications, reach out before the submission deadline.
6. Deadline:
    a. Ensure that you submit your lab on time, before the deadline. Late submissions will not be accepted
    b. Double-check that the zip file is correctly named and contains all necessary files before submitting.
7. Plagiarism Policy:
    a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

The Bare Minimum

A basic C++ program usually looks like this:

File: simple.cpp

```cpp
// Preprocessor Directives

int main() {
    // Code...
}
```

This is a completely valid program. If you try to run it, nothing will happen, but it will compile successfully. From this we learn an important point: the only thing the compiler really demands is the ***main*** function. Program execution always begins with ***main***, and without it no program can compile.

Preprocessor Directives

Although the bare minimum compiles, it does not do anything useful. To make the program perform meaningful tasks, we must provide it with access to additional functionality. This is done by giving instructions to the preprocessor.

A **preprocessor directive** is a command that the compiler handles before the actual compilation begins. One of the most common is the ***#include directive***, which instructs the compiler to copy the contents of another file into the program. By doing this, the compiler learns about new functions, objects, and definitions.

File: simple.cpp

```cpp
#include <iostream>

int main() {
    // Code...
}
```

The line ***#include <iostream>*** tells the compiler to include the header file named ***iostream***. When the compiler sees this directive, it *effectively* pastes the contents of ***iostream*** into the program. This header provides many of the most basic tools of C++, including the standard input and output streams. The angle brackets **<>** indicate that the header should be searched for in the *compiler's standard library directories.*

Printing with `std::cout`

One of the most useful objects defined in ***iostream*** is ***std::cout***. This object represents the standard output stream, which usually means the console window. To send data into this stream, we use the insertion operator **<<**.

If we extend our program to use it, the code becomes:

File: simple.cpp

```cpp
#include <iostream>

int main() {
    std::cout << "Hello, World!" << std::endl;
}
```

When compiled and run, this program displays the text "`Hello, World!`" on the screen. If we want the output to end with a newline, we can add either ***\n*** inside the string or attach ***std::endl*** at the end. For example:

```cpp
std::cout << "Hello, World!" << std::endl;
```

The ***std::endl*** manipulator inserts a newline character and flushes the output buffer, making sure the text appears immediately.

Compiling and Running the Program

The process of writing, compiling, and running the program can be done directly from the command line, without the need for a special editor.

1. First, create a new folder on your **Desktop** and open it.
2. In **File Explorer's** address bar, type **cmd** and press Enter. This opens Command Prompt in the folder you are currently in. *(Current Directory, or cwd – Current working directory)*
3. Next, type the command:

```
notepad hello.cpp
```

   This will create a new file named *hello.cpp* (if it doesn't exist) and open it in Notepad.

4. Write the code into this file and save it.
5. Back in Command Prompt, compile the file using:

```
g++ hello.cpp -o hello.exe
```

6. If the code is free of errors, the compiler will produce an executable named *hello.exe*. You can run it by typing:

```
hello.exe
```

If there is a mistake, such as forgetting a semicolon after a statement, the compiler will print an error message showing the line number and the nature of the problem.

Escape Sequences

When printing text, sometimes we need special characters that cannot be typed directly, such as newlines or quotation marks inside a string. For this purpose, C++ provides escape sequences. These are written as a backslash followed by another character.

- `\n` moves the cursor to a new line.
- `\t` inserts a tab.
- `\r` returns the cursor to the beginning of the same line.
- `\b` moves one character backward (backspace).
- `\\` prints a backslash.
- `\"` prints a double quote.
- `\'` prints a single quote.

For example, the following code uses `\n` to print two lines of output:

```
std::cout << "Hello, World!\n";
std::cout << "Welcome to C++ Programming\n";
```

The console output will be:

```
Hello, World!
Welcome to C++ Programming
```

Lab Tasks

Task 1: Introduction

Write a program that prints an introduction to yourself:

```
Hello! This is <your name>.

I am currently enrolled in <Program/Degree> at <University Name>.
The following topics interest me very much:

    1. <Interest 1>
    2. <Interest 2>
    3. ... at least 2 interests

Regards,
<Your ERP>
```

```
Intro-To-Programming-CPP/Lab_Solutions/Lab01
❯ g++ T1.cpp && ./a.out
Hello! This is Ammar.

I am currently enrolled in BSCS at IBA.
The following topics intrest me very much:
        1. Data Science
        2. Machine Learning

Regards,
12345
```

Use **\n** and **\t** to format the text neatly. You must use only one **std::cout** statement to print the entire introduction.

Task 2: Printing *ABC* without printing *ABC*

Write a program that demonstrates the use of **\r** (carriage return) by first printing **CCC** and then overwriting it with **BB**. Finally, overwrite **BB** with **A**.

Output Workflow:

1. Print **CCC**.
2. Use **\r** to move the cursor to the beginning of the line.
3. Print **BB** to overwrite **CCC**.
4. Use **\r** to move the cursor to the beginning of the line.
5. Print **A**.

```
Intro-To-Programming-CPP/Lab_Solutions/Lab01
❯ g++ T2.cpp && ./a.out
ABC
```

Task 3: Processing a file

Write a program that demonstrates the use of both **\n** and **\r**.

Output Workflow:

- Print ***Processing File...***.
- Use **\r** to move the cursor to the beginning of the line.
- Print ***Processing Complete!*** to overwrite ***Processing File...***.
- Print ***Done*** in a newline.

```
Intro-To-Programming-CPP/Lab_Solutions/Lab01
❯ g++ T3.cpp && ./a.out
Processing Complete!
Done
```

Task 4: Student Attendance Sheet

Write a program that demonstrates the use of **`\n`**, **`\t`**, and **`\r`**. The program should:

- Print a student attendance sheet in a tabular format using **`\t`**.
- Use **`\n`** to move between lines.
- Use **`\r`** to overwrite a temporary status ("Updating...") with the final status ("Update Complete!") on the same line.

```
Intro-To-Programming-CPP/Lab_Solutions/Lab01
❯ g++ T4.cpp && ./a.out

Name            Roll No.        Status
Ali             101             Present
Ahmed           102             Absent

Update Complete!
```

Output Workflow:

1. Print the table header:
   Name Roll No. Status
2. Print each student with roll number and attendance status:
   Ali 101 Present
   Ahmed 102 Absent
3. Print a temporary status message:
   Updating...
4. Use **`\r`** to return to the beginning of the line and overwrite the temporary message with the final one:
   Update Complete!

Task 5: Initializing Variables and Displaying them

Write a program that assigns values to 4 variables and displays a message using those variables. The program should:

- Contain 4 variables: *name, erp, section, course*
- Store releavant info in these variables
- Use **cout** to display the message

```
Intro-To-Programming-CPP/Lab_Solutions/Lab01
❯ g++ T5.cpp && ./a.out
My name is Ammar.
My ERP is 12345.
I am in section 99397.
I am studying Intro to Programming.
```

Task 6: Combining Variables and Sqrt

Many of you maybe asking what is *sqrt*?
sqrt() is a function in the <*cmath*> library. It means to take the square root of the integer given in the parenthesis *( )*
For example:
sqrt(49) is equal to 7.

Hint: #include<cmath>

Write a program that assigns a variable an integer then it passes that integer to the *sqrt()* function which will give us the square root of the number. This program should:

- Assign one variable an integer
- Display the square root of that integer

```
Intro-To-Programming-CPP/Lab_Solutions/Lab01
❯ g++ T6.cpp && ./a.out
Number: 144
Square Root: 12
```

Output Workflow:

1. Display the number
2. Display the square root of that number